

Chapter 5

Working with User Proxies

It is vital that a project include one or more real users on the customer team. While others may be able to guess at how a user wants the software to behave, only a real user knows. Unfortunately, it is often difficult to get the users we need. For example, we might be developing a shrinkwrap product with users across the country but be unable to bring one (or more) of them onsite with us to write the stories. Or we might be writing software that will be used within our company, but someone tells us we cannot talk to the users. When we cannot get as many users as we want to represent different perspectives of the product, we need to resort to *user proxies*, who may not be users themselves but are on a project to help represent users.

Selection of appropriate user proxies can be critical to the success of the project. The background and motives of possible user proxies must be considered. A user proxy with a marketing background will approach the stories differently than will a user proxy who is a domain expert. It is important to be aware of these differences. In this chapter we will consider various user proxy types who may sometimes fill in for real users.

The Users' Manager

When doing development on a project for internal use, the organization may be reluctant to give you full and unlimited access to one or more users but may be willing to give you access to the users' manager. Consider this a bait-and-switch unless the manager is also a true user of the software. Even then, it is almost certain that the manager has different usage patterns of the software than does a typical user. For example, on one call center application the team was initially given access to shift supervisors. While shift supervisors did use the software, many of the features they wanted in a new version were focused around managing call queues and transferring calls between agents. These features were of

very minimal importance to the users they supervised, for whom the software was mostly intended. If the developers had not pushed for direct access to more typical users, the supervisors' less frequently used features would have been overemphasized in the product.

Sometimes the users' manager intercedes and wants to play the user role on the project because of her ego. She may acknowledge that she's not a typical user but will insist that she knows more about what her users need than they do. Naturally, though, in this type of situation you will need to be careful not to offend the user's manager. But you do need to find a way at least partially around her and to the end users for the project to succeed. Some ideas for this are given later in this chapter in the section "What to Do When Working with a User Proxy" on page 61.

Five Minutes Does Not Equal One Minute

The "user" on this internal project was a Vice President who never used the software and had a level of managers between her and the end users. In prioritizing stories for the next iteration, she wanted the developers to focus on improving the speed of the database queries. The team noted the story and its high priority but they were puzzled. They knew application performance was critical and had built a monitoring mechanism into the software: each time a database query was executed, its parameters, the time it took to execute, and the name of the user were stored in the database. This information was monitored at least once a day and there had been no indications of a performance problem. Yet their "user" had told them that some queries were taking "up to five minutes."

After the meeting with the Vice President, the team looked into the query execution history. Here's what they found: A couple of users had in fact executed queries that took one minute to complete. That was certainly longer than desired but considering what they were searching for, the size of the database, and the infrequency of that type of search, it was within the expected performance of the system. But the users had reported the one-minute query to their manager. The manager then reported it to the Vice President; but to make sure the problem got the Vice President's attention, the manager said queries were taking two minutes. Then the Vice President reported it to the developers, and to make sure she got their attention, increased the problem to "up to five minutes."

Users' managers can be sources of misinformation. Whenever possible, corroborate their statements by talking to real users.

A Development Manager

A development manager is one of the worst possible choices to act as a proxy user, unless perhaps you are writing software targeted at development managers. While the development manager may have nothing but the most honorable intentions, it is far too likely that she will also have some conflicting goals. For example, the development manager may prioritize stories differently than would a real user because doing so allows her to accelerate the introduction of an exciting, new technology. Additionally, the development manager may have unaligned corporate goals: perhaps her annual bonus is tied to a completion date on the project, which could cause her to consider the project complete before a real user would.

Finally, most development managers simply do not have hands-on experience as users of the software they are building and are not domain experts. If your prospective user is a development manager who does have domain expertise, then consider her a domain expert and read the discussion in the “Domain Experts” section of this chapter before deciding if you have an adequate user proxy.

Salespersons

The danger in using a salesperson as a user proxy is that it does not lead to a comprehensive view of the product to be built. The most important story to a salesperson will usually be the story whose absence cost her the last sale. If she lost a sale because the product does not have an undo feature, you can bet that the undo story card will be instantly sorted right to the top of the pile. Depending on the importance of a specific lost sale, it may be desirable to write a new story or two; however, a product development company that puts too much emphasis on each lost sale may lose track of whatever strategic, long-term vision is held for the product.

Salespeople are, however, a great conduit to users and you should use them in this way. Ask them to introduce you to customers either on the phone or while along on a sales visit. Even better, attend an industry trade show and work in your company's exhibit.

Talk to Your Users

In 1995 this team was challenged with creating what would be one of the web's first general health information sites. Because there were not yet any competitors, the team could not look to them for story ideas. The user proxy for the project was a director with a marketing background. Because of his marketing background, he understood the importance of talking with prospective users to find out what they would want in a health information site. However, because of the pressure to deliver the site quickly, he plunged forward and built the site using only his gut feel to guide him.

As you can guess, the project didn't fulfill its users' needs. About a month after the site launched I walked into the marketing director's office. He pointed to his monitor and said, "Look at that. Just look at that." On his screen was a porn site. After a moment I asked why we were looking at it. I don't think he'd even noticed the porn on the site. His eyes were fixed on a hit counter and he said, "Just look, they've had 100,000 hits since yesterday. We've had 200."

If you want your software to be used, you have to talk to those who will use it.

Domain Experts

Domain experts, sometimes called subject matter experts, are critical resources because of how well they understand the domain the software will be targeted at. Naturally some domains are harder to understand than others. I used to write a lot of software for attorneys and paralegals, and while the software was sometimes complex, I could usually understand what they were asking for. Much later, I was involved with writing software for statistical geneticists. This domain was filled with words like phenotype, centimorgan, and haplotype. These were words I had never heard before, which made the domain much harder to grasp. This made each of the developers much more reliant on a domain expert to help us understand what we were developing.

While domain experts are great resources, their usefulness is really dependent upon whether they are current or former users of the software type you are building. For example, when building a payroll system you will undoubtedly want to have a Certified Public Accountant (CPA) available as a domain expert. However, since the users will probably be payroll clerks and not CPAs, you will probably get better stories from the payroll clerks. Domain experts are ideal resources when building a domain model and identifying business rules, but workflow and usage issues are better derived from actual users.

Another potential problem with using a domain expert as your user proxy is that you may end up with software aimed only at users with similar levels of domain expertise. Domain experts can be inclined to point the project toward a solution that is suitable for them but is too complex or is just plain wrong for the targeted user audience.

The Marketing Group

Larry Constantine and Lucy Lockwood (1999) point out that marketing groups understand markets rather than users. This can lead a marketing group, or someone with a marketing background, to focus more on the quantity of features in the product than on the quality of those features. In many cases a marketing group may provide useful high-level guidance about relative priorities, but often does not have the insight to provide specific details about the stories.

Former Users

A former user can be great as a proxy if her experience is very recent. However, as with other user proxies, you should carefully consider whether the former user's goals and incentives are fully aligned with those of the real users.

Customers

Customers are those who make the buying decision; they are not necessarily users of the software. It is important to consider the desires of your customers because they, not your users, are the ones who write the check to buy the software. (Unless, of course, your users and customers are the same people.)

Corporate desktop productivity software is a perfect example of the distinction between customer and user. The corporate IT staff may decide which word processor is used by all the employees of the company. In this case, the IT staff is the customer but all employees of the company are users (including the IT staff, who are both customer and user). The features in a product like this must be sufficient that the users do not scream too loudly; but the features must also be those that appeal to the customer making the buying decision.

Talk to Your Users Redux

In one company the marketing group was serving as a customer proxy for a new product that would replace the company's current paper-based product. The company had a very successful history selling printed books containing rules that hospitals and insurance companies had agreed upon: if the hospital followed the rules they would be reimbursed by the insurance company. For example, an appendectomy was only called for if (among other things) a patient's white blood count was above a certain threshold.

The marketing group had no interest in talking to the users of the printed books to find out what they might want the software to do. Instead they reasoned that they knew exactly what their users would want and development could proceed under the guidance of the marketing group. The marketing group chose a book metaphor for the software. Rather than take advantage of the inherent flexibility of software, they settled for an "automated book." The users were, quite obviously, disappointed with the software. Unfortunately, the company could have known this very early on if they had used real users rather than a marketing department as a user proxy.

For example, security features are typically unimportant to most users of desktop productivity software. Security is vitally important, however, to the IT staff (the customers) who make the buying decision.

One project team I worked with had designed a database-intensive application. Data would be loaded into the system from other systems the customers already had. The developers needed to specify a file format that would be used to exchange this data. In this case the customer was the CIO of the organization; the users of this functionality were the IT staff in his organization who would write the extract program to move data from their current systems into the format specified for the new system. When asked about his preferences for the file format, the customer (the CIO) decided that XML would be an ideal technology since it was relatively new at the time and was certainly sexier than a non-standard comma-separated value (CSV) file. When the software was delivered, the users (the IT staff) completely disagreed—they would have preferred the much simpler-to-generate CSV file over the XML file. If the development team had got the stories directly from the users, they could have known this and would not have wasted time on the XML format.

Trainers and Technical Support

Trainers and technical support personnel may seem like logical choices to fill in as a user proxy. They spend their days talking to real users so they must certainly know what users want. Unfortunately, if you use a trainer as your user proxy, you will end up with a system that is easy to train. Similarly, if you use someone from technical support, you will end up with a system that is easily supported. For example, someone from technical support may put low priority on the advanced features that she anticipates will lead to increased support work. While training ease and supportability are good goals, they are most likely not what a true user would prioritize.

Business or Systems Analysts

Many business and systems analysts make good user proxies because they have one foot in the technology world and one foot in the domain of the software. An analyst who can balance these backgrounds and who puts effort into talking with actual users is often an excellent user proxy.

One problem that some analysts exhibit is that they prefer to think about a problem rather than research it. I've worked with too many analysts who believe they can sit in their offices and intuit what users will want without actually talking to those users. Be watchful that the project's analyst talks to users and that she doesn't just make decisions on her own.

A second problem I've occasionally encountered with analysts is a desire to spend too much time in the project's upfront activities. So, where a two-hour role modeling and story writing workshop might be sufficient to fill in details for a release plan extending out four months, some analysts will prefer to instead spend three weeks on these activities.

What to Do When Working with a User Proxy

Although not ideal, it is still possible to write great software with a user proxy rather than a real user. There are a number of techniques you can apply to increase your chances of success in these cases.

When Users Exist But Access Is Limited

If access to real users is blocked and the team is instead told to work through a user proxy who will make all project decisions, the team will need to work with the proxy but also establish a connection to hands-on users. One of the best techniques for doing this is to request permission to start a *user task force*. A user task force consists of any number of real users, from a handful to a couple of dozen. The task force is used as a sounding-board for ideas, but the proxy remains the project's final decision-maker. In most cases the proxy will go along with this, especially as it gives her a safety net to protect her against bad decisions.

Once a user task force is established and staffed with real users, it can typically be used to guide more and more of the day-to-day decisions on the project. You can do this by having a series of meetings to discuss parts of the application and then have the task force identify, write, and prioritize stories.

One project team I worked with was developing a system for internal users and had great success by taking top-level direction from a user proxy, using prototypes to show a user task force, and then acting on feedback generated during user task force meetings. This particular project ran with month-long iterations. The first few days to a week of each iteration was spent prototyping and holding one or more user task force meetings. In this way the user proxy (the users' manager in this case) had fine control over the strategic direction of the project, but the implementation details were moved from her to the user task force.

When There Really Is No User Available

When there really is no user available and you must resort to a user proxy, one valuable technique is to use more than one user proxy. This helps reduce the likelihood of building a system that meets exactly one person's needs. When using more than one user proxy, be sure to use different types of user proxy. For example, combine a domain expert with someone from marketing rather than using two domain experts. You can do this by either having two designated user proxies, or by having one designated user proxy but encouraging her to rely on other, informal user proxies.

If you are developing software that will compete with other commercial products, you can use the competing products as a source for some stories. What features in the competing products get mentioned in reviews? What features are discussed in online newsgroups? Are the features discussed because they are overly complex to use?

I remember arguing a few years ago with a use case advocate about what type of document best expresses the requirements of a system. He argued in favor of a well thought-out use case model. I argued in favor of the user's guide. I have never seen a project that concluded with a perfectly accurate and current use case model even though some have tried. I have seen many projects that conclude with an accurate and current user's guide. If you are writing new software that will compete with existing products, you can learn a great deal by studying the competing products.

Another technique you can use when working with a user proxy rather than real users is to release the product as soon as possible. Even if the release is called preliminary or an early beta, getting it into the hands of users early will help identify inconsistencies between the thinking of your user proxy and your real users. Even better, once the software is in the hands of one or more early adopters, you have now opened up a communication path to those users and can use that to talk with them about upcoming features.

Can You Do It Yourself?

When you cannot find, or get access to, a real user, avoid falling into the trap of thinking you know your users' minds and do not need or can ignore your user proxy. While each type of user proxy has some type of shortcoming that makes her less desirable than a real user, most developers come with even more shortcomings for pretending to be a real user. In general, developers do not have marketing backgrounds that allow them to understand the relative value of features, they do not have the same amount of customer contact as salespeople, they are not domain experts, and so on.

Constituting the Customer Team

First, always remember that a real user beats a proxy any time. Whenever possible place real users on the customer team. However, when you can't get the right mix of real users, supplement the customer team with one or more user proxies. The customer team should be constructed so that the strengths of one member balance the weaknesses of another member. There are four steps in creating a customer team.

First, add real users. If the software is used by different types of users, try to include a user of each type. For example, on one health care application our

users were nurses. On our customer team we had users who were regular nurses, oncology specialists, diabetic specialists, and so on.

Second, identify a single project champion or “first among equals” on the customer team. In commercial software companies this is very often a product manager, but it could be someone else. This project champion becomes responsible for coordinating customer team collaboration. All members of the customer team are, to the extent they can achieve, responsible for delivering a consistent message. While there may not be one customer, there must be one customer voice.

Third, determine the factors critical to project success. This will vary from project to project. For example, if the project is to create a next generation version of an existing product, then a critical success factor will be how easily existing users can move to the new system. Supplement the customer team with user proxies with the relevant knowledge, skills, and experience to address the project’s critical success factors. In our example of moving existing users to a new system, this might mean adding a trainer to the customer team.

Summary

- In this chapter we learned about different types of user proxies and why no user proxy is as ideal as a true user when it comes to writing *user* stories.
- The users’ manager may not be an appropriate user proxy unless she is also a user.
- Development managers make tempting user proxies because they are already involved in the day-to-day detail of the project. However, the development manager is rarely an intended user of the software being built and is therefore a poor choice as a user proxy.
- In product companies, the customer frequently comes from the marketing group. Someone from the marketing group is often a good choice as a user proxy but must overcome the temptation to focus on the quantity rather than quality of features.
- Salespeople can make good customers when they have contact with a broad variety of customers who are also users. Salespeople must avoid the temptation to focus on whatever story could have won the last lost sale. In all cases, salespeople make excellent conduits to users.

- Domain experts can make excellent user proxies but must avoid the temptation to write stories for a product that only someone with their expertise can use.
- Customers, those who make the purchasing decision, can make great user proxies if in close communication with the users for whom they are purchasing the software. Obviously, a customer who is also a user is a fantastic combination.
- In order to be good user proxies, trainers and technical support personnel must avoid the temptation to focus too narrowly on the aspects of the product they see every day.
- This chapter also looked briefly at some techniques for working with user proxies, including the use of user task forces, using multiple user proxies, competitive analysis, and releasing early to get user feedback.

Developer Responsibilities

- You have the responsibility to help your organization select the appropriate customer for the project.
- You are responsible for understanding how different types of user proxies will think about the system being built and how their backgrounds might influence your interactions.

Customer Responsibilities

- If you will not be a user of the software, you are responsible for knowing which categories of user proxy describe you.
- You are responsible for understanding what biases you may bring to the project and for knowing how to overcome them, whether by relying on others or some other means.

Questions

- 5.1 What problems can result from using the users' manager as a proxy for the users?
- 5.2 What problems can result from using a domain expert as a proxy for users?